



2^{ème} partie

On veut maintenant ajouter le contrôle suivant. Avant qu'un utilisateur modifie son mot de passe, l'application s'assure que son mot de passe actuel correspond bien à celui déjà stocké dans l'objet user.

1. Quel est le meilleur endroit pour effectuer cette vérification de mot de passe ?
2. Décrire les étapes à faire pour ajouter cette vérification de mot de passe (Création nouvelle classe, modification du fichier **web.xml**...)
3. Ecrire le code nécessaire permettant de rendre obligatoire la vérification du mot de passe avant de le modifier.

الاتحاد الوطني لطلبة
U.N.E.M. موريتانيا

l'intégralité des données contenues dans la table employe

EXERCICE II

1. Vous connecter en tant qu'utilisateur disposant du rôle « sysdba »
2. Créer un compte « stage » disposant d'un mot de passe aussi « stage ».
3. Accorder au compte créé précédemment les droits et les rôles suivants :
Connect, create session et resource.
4. Retirer du même compte créé déjà les mêmes droits et rôles qui lui sont déjà affectés récemment.

BONNEN CHANCES

الاتحاد الوطني لطلبة
U.N.E.M. موريتانيا



```
private Map<Integer, Admin> memoireadmin = Donnees.getAdmin();
```

-----get all-----

```
public List<Admin> getAdmins() {  
    return new ArrayList<Admin>(memoireadmin.values());  
}
```

-----get one-----

```
public Admin getAdmin(Integer id) {  
    return memoireadmin.get(id);  
}
```

-----create one-----

```
public Admin addcaiss(Admin adm) {  
    memoiradmin.setIdCaiss(admin.size() + 1);  
    memoiradmin.put(admin.getIdAdmin(), adm);  
    return adm;  
}
```

-----update one-----

```
public Admin UpdateAdmin(Admin admin) {  
    if (admin.getUsername() != null) {  
        memoiradmin.put(1, admin);  
        return admin;  
    }  
    throw new RuntimeException("il n ya aucun admin de modifier");  
}
```

-----delete one-----

```
public Admin deleteadmin(Integer id) {  
    return memoiradmin.remove(id);  
}
```

-----Controllor-----

```
ServiceAdmin s = new ServiceAdmin();
```

```
@Path("/admin")
```

```
@Produces("application/json")
```

```
@Consumes(MediaType.APPLICATION_JSON)
```

```
public class Admincontroler {
```



@GET

@Path("/listadmin")

```
public List<Admin> listadmin() {  
    return s.getAdmins();  
}
```

-----Memoire DB-----

```
public class Donnees {  
    private static Map<Integer, Admin> memoireadmin = new HashMap<>();  
    public static Map<Integer, Admin> getMemoireadmin() {  
        # memoireadmin.put(1, new Admin("admin", "admin"));  
        return memoireadmin;  
    }  
}
```

-----Filter-----

```
public class AdminFilter implements Filter {  
    public void init(FilterConfig fConfig) throws ServletException {  
    }  
    public void doFilter(ServletRequest request, ServletResponse response, FilterChain chain)  
        throws IOException, ServletException {  
        stringunzip(msg);  
        HttpServletRequest httpReq = (HttpServletRequest) request;  
        HttpSession session = httpReq.getSession();  
        if (session.getAttribute("dgSession") != null || session.getAttribute("adminSession") != null  
            || session.getAttribute("ssesioncaiss") != null) {  
            // sends request to next resource  
            chain.doFilter(request, response);  
        } else {  
            PrintWriter out = response.getWriter();  
            out.print(" Seul l'admin peut acceder a cette page");  
        }  
    }  
    public void destroy() {  
        // TODO Auto-generated method stub  
    }  
}
```